

Proiectarea sistemului de operare în timp real MyOS Resursele alocate unui task. Administratorul de task-uri

1. Obiectivul lucrării Conform observațiilor făcute în laboratoarele anterioare, un task alături de spațiul alocat în memoria de program pentru corpul funcției propriu-zis, are nevoie în plus și de resurse de memorie RAM unde să își salveze contextul: setul de variabile pe care acesta îl folosește și stiva cu apeluri de funcții. Pe de altă parte, după cum a fost prezentat în cursul alocat planificării proceselor, administratorul de task-uri alocă la fiecare task nivelul său de prioritate, starea acestuia și timpul de comutare. Pentru toate aceste informații, caracteristice unui task, trebuie alocat un anumit spațiu de memorare. Pentru aceasta este nevoie de efectuarea unor calcule preliminare cu resursele hardware pe care le necesită fiecare task în parte. În acest laborator se va prezenta modul în care este descris un task și determinarea resurselor de memorie alocate pentru acesta pentru ca acesta să fie administrat.

2. Breviar teoretic

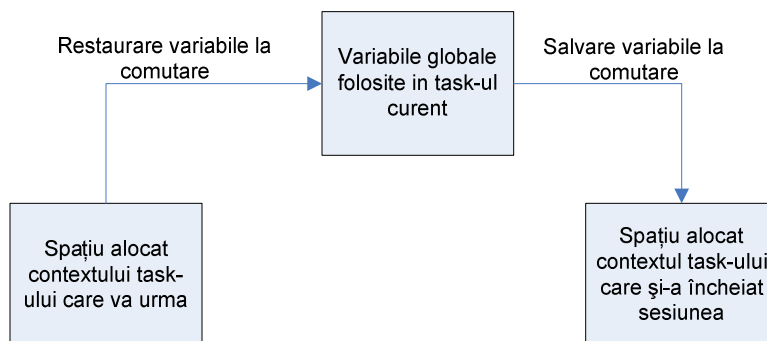
Un task, sintetizând cele menționate mai sus, se poate descrie ca o structură de felul următor:

```
struct task{  
    unsigned char context[DIM_CONTEXT];  
    unsigned char prioritate;  
    unsigned char stare;  
    unsigned char timp_alocat;  
};
```

Contextul

Reprezintă setul de variabile și stiva hardware alocată pentru task-ul respectiv. După cum s-a prezentat în laboratorul anterior un mod tipic de lucru cu variabile este următorul:

- se definește un set de variabile globale curente, accesibil task-ului curent;
- la comutarea între două task-uri, variabilele globale tipice task-ului care părăsește sesiunea de execuție sunt salvate, și se restaurează variabilele task-ului care va intra în sesiunea de execuție (task curent).



Pentru variabilele globale definite a, b, c secvența de comutare descrisă folosind limbajul C arată în felul următor:

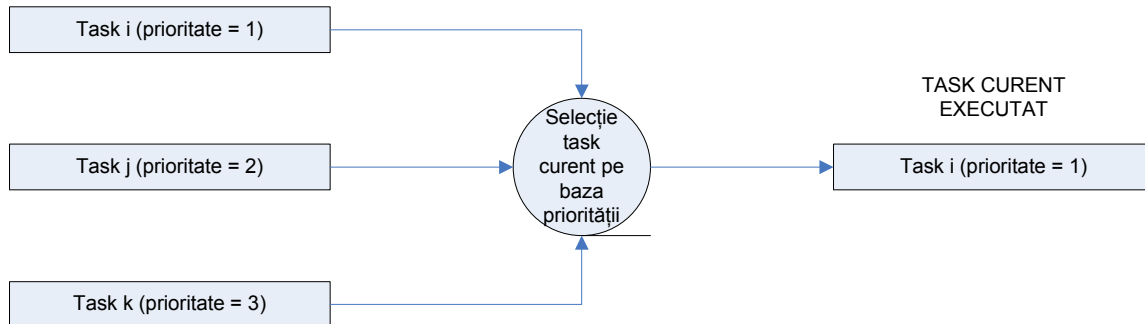
```
//comuta
//salvare adrese task care a parasit sesiunea
context[task_curent-1][0]=a;
context[task_curent-1][1]=b;
context[task_curent-1][2]=c;

//restaurare adrese task care se va executa
a=context[task_curent][0];
b=context[task_curent][1];
c=context[task_curent][2];
```

Alături de variabilele globale, este posibil ca fiecare task să aibă propriul set de variabile. Acesta nu este necesar să fie salvat în context dar la definirea unei variabile „locale” accesibile unui task trebuie avut în vedere că aceasta va avea alocat în memorie un spațiu care se va aduna la cel alocat task-ului respectiv.

Prioritatea

După cum a fost prezentat la cursul 5 (Planificarea task-urilor), o metodă eficientă de a planifica task-urile este prin alocarea unui grad de prioritate. Astfel, dacă mai multe task-uri sunt pregătite de execuție se va executa task-ul cu gradul de prioritate cel mai mare. Prioritatea este definită ca un număr întreg pozitiv, invers proporțional cu gradul de prioritate: cel mai mic număr întreg pozitiv reprezintă cel mai mare grad de prioritate.



Planificarea pe baza priorităților nu este o metodă statică de planificare. Prioritățile se pot modifica în timpul operării, în funcție de cerințele sistemului la un moment dat. Din această cauză, la comutarea între task-uri întotdeauna trebuie determinat task-ul cu gradul cel mai mare de prioritate.

//determinare task curent avand trei task-uri “candidate”: task_i, task_j si task_k

```
if(task_i.prioritate<task_j.prioritate)
    task_curent = task_i;
else
```

```
task_curent = task_j;
```

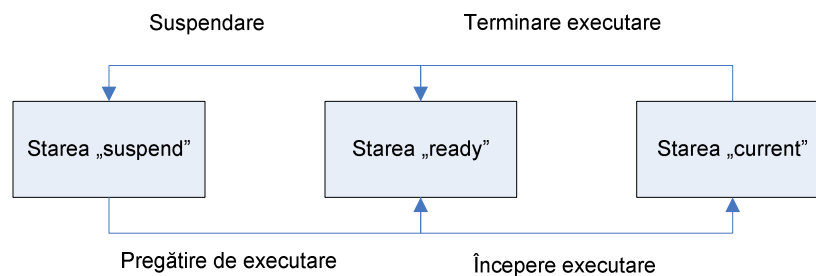
```
if(task_curent.prioritate > task_k.prioritate)  
    task_curent = task_k;
```

Starea

Reprezintă starea curentă a task-ului. Fiecare stare este codificată printr-o valoare. Avem definite următoarele stări: Ready (gata de execuție), Suspend (suspendat momentan), Current (task executat curent).

Fiecare stare este codificată printr-un cod ca în tabelul de mai jos:

Stare	Cod
Ready	01
Suspend	02
Current	00



La comutarea task-urilor se iau în considerare numai task-urile din starea Ready. Definirea stărilor se face astfel:

```
#define READY 1  
#define SUSPEND 2  
#define CURRENT 0
```

Apoi pentru selectarea task-ului curent se face testul task-ului candidat task_i:

```
if(task_i.stare == READY)  
    //comuta task_i pe task curent
```

Timpul alocat

Reprezintă timpul alocat pentru un task ca să fie executat. Acest timp este de regulă dat în unități de timpi elementare definite. De exemplu, dacă se ia ca unitate de timp elementară 1ms atunci dacă dorim pentru un task task_i să comute 1 un interval de 3 ms atunci se definește timpul alocat pentru acest task: $task_i.timp_alocat = 3$.

În momentul în care se dorește comutarea pe un task_i timpul de execuție se setează cu valoarea timp_alocat corespunzătoare:

```
timp_executie = task_i.timp_alocat;
```

În această situație, comutarea la task-ul următor se va realiza în subrutina de tratare a întreruperii provenite de la circuitul timer, subrutină care apare la o perioadă de timp elementar:

```
//subrutina apare la un interval de timp egal cu timpul elementar  
timp_executie--;  
if(timp_executie == 0)  
{  
    //comuta task urmator.
```

Pentru toate aceste date va trebui alocat spațiu un memorie pentru stocare. Deci cantitatea de memorie necesară unui task se calculează în funcție de dimensiunea acestor câmpuri din interiorul structurii task.

3. Desfășurarea lucrării

Lucrarea își propune rezolvarea mai multor probleme legate de sistemele în timp real.

1. Se consideră trei task-uri, cu următoarea organizare:

Task	Prioritate	Perioadă apariție (T=D)	Timp calcul
T1	1	7 ms	2 ms
T2	2	16 ms	4 ms
T3	3	31 ms	7 ms

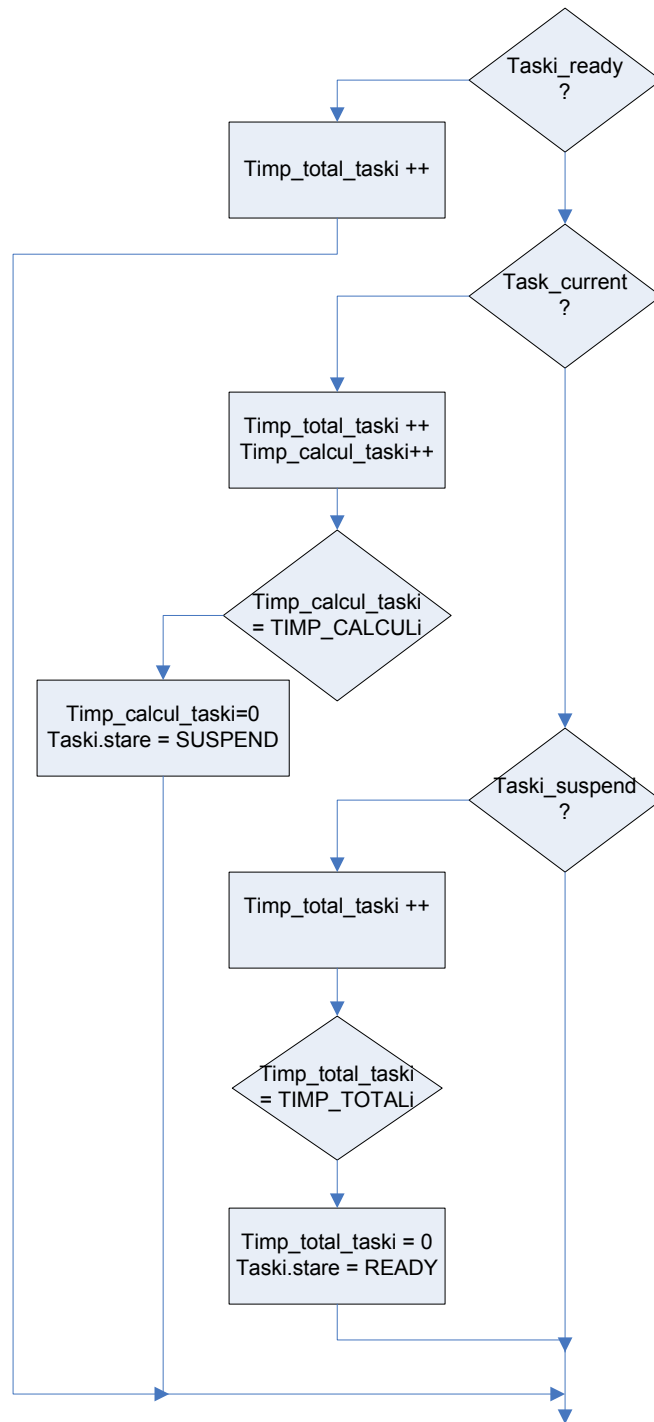
Să se realizeze task managerul și să se determine, prin rularea lor, dacă sistemul astfel organizat este fezabil, pentru un sistem cu ATMEGA128. Frecvența de lucru a sistemului este de 4MHz.

Rezolvarea problemei va conține mai multe etape. În primul rând, se va alege unitatea de timp elementară astfel încât toate unitățile de timp impuse să fie multipli din aceasta. Ne putem da seama ușor că unitatea de timp elementară trebuie să fie milisecunda. Pentru aceasta vom programa canalul timer să genereze întreruperi la 1ms. Task-urile se presupun toate inițial în starea ready (gata de execuție). După ce este executat un task complet (întreg timpul de calcul alocat lui) acesta este trecut în starea suspendat până la sfârșitul perioadei de apariție T). După ce trece perioada de apariție task-ul va trece din nou în starea ready.

Comutarea între task-uri se face pe baza criteriului de prioritate. Task-ul curent poate fi numai un task aflat în starea ready.

Pentru monitorizarea timpilor de execuție și a timpilor totali pentru a verifica corectitudinea implementării acestei probleme se definesc pentru fiecare task variabilele timp_total_taski și timp_calcul_taski. Când task-ul se află în starea ready sau în starea curentă se incrementează timpul total. Când task-ul se află în starea current timpul de

calcul și timpul total sunt incrementate. Cele două variabile se resetează când task-ul este suspendat, deci când acesta a fost tratat și a mai rămas un timp liber.



1. Configurarea hardware

În această problemă, este necesară realizarea unei temporizări de 1ms deoarece toți timpii impuși în cerințele problemei sunt multipli de milisecundă.

Având o frecvență de 4MHz și cunoscând din laboratoarele anterioare modul în care se poate configura circuitul temporizator avem:

Se alege un prescaler 32:

$$4\,000\,000 / 32 = 125\,000.$$

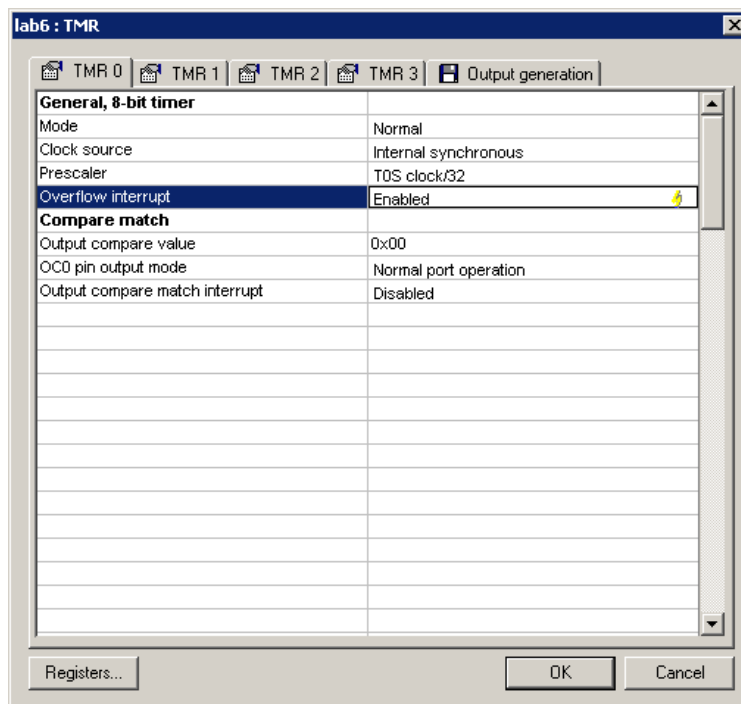
Apoi avem $125\,000 / 1000 = 125$. Deci divizarea canalului trebuie să fie cu 125.

Constanta de divizare a canalului este deci: $256 - 125 = 131$.

Prin urmare, la începutul oricărui apel de subrutină de tratare a întreruperii la canalul timer avem $TCNT0 = 131$.

Se activează lucrul cu întreruperi.

Pentru configurare se va utiliza MakeApplication.



2. Configurarea software

În primul rând se începe cu definirea variabilelor folosite în program. Pentru aceasta se construiește un fișier varaibile.c cu următorul conținut:

variabile.c

```
#include "variabile.h"
```

```
//variabila care semantizeaza aparitia unui eveniment - in acest caz comutare intre taskuri
```

```
unsigned char eveniment=0;
```

```
//numarul aparitiilor unui task - varaibila globala accesata de toate task-urile  
int numar_aparitii;
```

```
//timpi totali si timpi de calcul
```

```
unsigned char timptotal_task1=0;
```

```
unsigned char timpcalcul_task1=0;
```

```
unsigned char timptotal_task2=0;
```

```
unsigned char timpcalcul_task2=0;
```

```
unsigned char timptotal_task3=0;
```

```
unsigned char timpcalcul_task3=0;
```

```
//definire task-uri
```

```
task task1,task2,task3,task_crt;
```

Este definit spațiul ocupat de fiecare task în memoria RAM. Definirea tipului de date se face în fișierul variabile.h.

variabile.h

```
//definire stari
```

```
#define CURRENT 0
```

```
#define READY 1
```

```
#define SUSPEND 2
```

```
//definire timpi totali
```

```
#define T_TASK1 7
```

```
#define T_TASK2 16
```

```
#define T_TASK3 31
```

```
//variabila care defienste task-ul curent
```

```
extern unsigned char task_curent;
```

```
//variabila care semantizeaza aparitia unui eveniment - in acest caz comutare intre taskuri
```

```
extern unsigned char eveniment;
```

```
//numarul aparitiilor unui task - varaibila globala accesata de toate task-urile
```

```

extern int numar_aparitii;

//timpi totali si timpi de calcul
extern unsigned char timptotal_task1;
extern unsigned char timpcalcul_task1;

extern unsigned char timptotal_task2;
extern unsigned char timpcalcul_task2;

extern unsigned char timptotal_task3;
extern unsigned char timpcalcul_task3;

//definire tip task

typedef struct Task{
    int context;           //contextul salvat
    unsigned char stare;   //starea task-ului
    unsigned char prioritate; //gradul de prioritate
    unsigned char timp;     //timpul de executie (timpul de calcul)
}task;

//definire task-uri
extern task task1,task2,task3,task_crt;

```

Calculul spațiului alocat, pentru fiecare task, în memoria ram se face conform următoarelor formule:

Spațiu alocat = dimensiune(context) + 1 (stare) + 1 (prioritate) + 1 (timp executie).

În cazul de față contextul se reduce doar la o variabilă de tip întreg. Deci spațiul alocat va fi de 5 octeți.

Administratorul de task-uri precum și simulatorul sunt declarate în fișierul `ma_tmr.c` unde este declarată subrutina.

Adăugare fișier **ma_tmr.c**

```

#pragma vector=MA_TIMER0_OVF_vect
__interrupt void MA_IntHandler_OVF0_TMR( void )

{

    TCNT0 = 131;
    //contorizarea timpilor alocați pentru fiecare task

```

```

switch(task1.stare)
{
case READY:
    timptotal_task1++;
    break;
case CURRENT:
    timptotal_task1++;
    timpcalcul_task1++;
    if(timpcalcul_task1==task1.timp)
    {
        timpcalcul_task1=0;
        task1.stare=SUSPEND;
    }
    break;
case SUSPEND:
    timptotal_task1++;
    if(timptotal_task1==T_TASK1)
    {
        timptotal_task1=0;
        task1.stare=READY;
    }
    break;
}

```

```

switch(task2.stare)
{
case READY:
    timptotal_task2++;
    break;
case CURRENT:
    timptotal_task2++;
    timpcalcul_task2++;
    if(timpcalcul_task2==task2.timp)
    {
        timpcalcul_task2=0;
        task2.stare=SUSPEND;
    }
    break;
case SUSPEND:
    timptotal_task2++;
    timpcalcul_task2=0;
    if(timptotal_task2==T_TASK2)
    {
        timptotal_task2=0;
        task2.stare=READY;
    }
}

```

```

    break;
}
switch(task3.stare)
{
    case READY:
        timptotal_task3++;
        break;
    case CURRENT:
        timptotal_task3++;
        timpcalcul_task3++;
        if(timpcalcul_task3==task3.timp)
        {
            timpcalcul_task3=0;
            task3.stare=SUSPEND;
        }
        break;
    case SUSPEND:
        timptotal_task3++;
        timpcalcul_task3=0;
        if(timptotal_task3==T_TASK3)
        {
            timptotal_task3=0;
            task3.stare=READY;
        }
        break;
}

```

```

//salvare context task curent
if(task1.stare==CURRENT)
    task1.context = numar_aparitii;
if(task2.stare==CURRENT)
    task2.context = numar_aparitii;
if(task3.stare==CURRENT)
    task3.context = numar_aparitii;

```

//calcul task curent: task-ul aflat in starea ready sau current cu gradul de prioritate cel mai mare

```

if((task1.stare!=SUSPEND)&&(task2.stare!=SUSPEND)&&(task3.stare!=SUSPEND))
{
    if(task1.prioritate<task2.prioritate)
    {
        if(task1.prioritate<task3.prioritate)
        {
            task1.stare = CURRENT;

```

```

    task2.stare = READY;
    task3.stare = READY;
}
else
{
    task3.stare = CURRENT;
    task2.stare = READY;
    task1.stare = READY;
}
}
else
{
    if(task2.prioritate < task3.prioritate)
    {
        task1.stare = READY;
        task2.stare = CURRENT;
        task3.stare = READY;
    }
    else
    {
        task1.stare = READY;
        task2.stare = READY;
        task3.stare = CURRENT;
    }
}
} //if toate 3 ready
//doar doua pregatite de a executa
else{

    if((task1.stare != SUSPEND) && (task2.stare != SUSPEND))
    {
        if(task1.prioritate < task2.prioritate)
        {
            task1.stare = CURRENT;
            task2.stare = READY;
        }
        else
        {
            task2.stare = CURRENT;
            task1.stare = READY;
        }
    }
    if((task2.stare != SUSPEND) && (task3.stare != SUSPEND))
    {
        if(task2.prioritate < task3.prioritate)
        {

```

```

        task2.stare=CURRENT;
        task3.stare=READY;
    }
    else
    {
        task3.stare=CURRENT;
        task2.stare=READY;
    }
}
if((task1.stare!=SUSPEND)&&(task3.stare!=SUSPEND))
{
    if(task1.prioritate<task3.prioritate)
    {
        task1.stare=CURRENT;
        task3.stare=READY;
    }
    else
    {
        task3.stare=CURRENT;
        task1.stare=READY;
    }
}
} //doar doua pot fi executate
//restaurare context task curent
if(task1.stare==CURRENT)
    numar_aparitii = task1.context;
if(task2.stare==CURRENT)
    numar_aparitii = task2.context;
if(task3.stare==CURRENT)
    numar_aparitii = task3.context;

eveniment = 1; //declansare eveniment

}

```

Programul principal și declararea task-urilor se face în fișierul main.c.

main.c

```

//incluziuni
#include<inavr.h>
#include<stdio.h>
#include"ma_tmr.h"
#include"variabile.h"

void functie_task(void)

```

```

{
    for(;;)
    {
        while(task1.stare==CURRENT)
        {
            if(eveniment)
            {
                MA_Stop_TMR(0);
                eveniment = 0;
                printf("Executa task
1| %d| %d| %d\n", timpcalcul_task1, timptotal_task1, numar_aparitii);
                numar_aparitii++;
                MA_Start_TMR(0,0);
            }
        }
        while(task2.stare==CURRENT)
        {
            if(eveniment)
            {
                MA_Stop_TMR(0);
                eveniment = 0;
                printf("Executa task
2| %d| %d| %d\n", timpcalcul_task2, timptotal_task2, numar_aparitii);
                numar_aparitii++;
                MA_Start_TMR(0,0);
            }
        }
        while(task3.stare==CURRENT)
        {
            if(eveniment)
            {
                MA_Stop_TMR(0);
                eveniment = 0;
                printf("Executa task
3| %d| %d| %d\n", timpcalcul_task3, timptotal_task3, numar_aparitii);
                numar_aparitii++;
                MA_Start_TMR(0,0);
            }
        }
    }
}

//functia principala
void main(void)
{
    //initializare timer

```

```

    MA_Init_TMR();
//pornire timer
    MA_Start_TMR(0,1);
//afisare mesaj intampinare
    printf("Operatie\Tcalcul crt.\Timp total crt.\Aparitie\n");
//initializare task-uri conform datelor problemei
//initializare task 1
    task1.context=0;
    task1.stare=READY;
    task1.prioritate=1;
    task1.timp=2;
//initializare task 2
    task2.context=0;
    task2.stare=READY;
    task2.prioritate=2;
    task2.timp=4;
//initializare task 3
    task3.context=0;
    task3.stare=READY;
    task3.prioritate=3;
    task3.timp=7;

//activare intreruperi
    __enable_interrupt();
//pornire task
    functie_task();
}

```

1. Care este dimensiunea în octeți a contextului în problema de mai sus pentru fiecare task?
2. Care este dimensiunea în octeți a spațiului alocat în memoria RAM din problema de mai sus pentru fiecare task în parte?
3. Care este dimensiunea în octeți alocată în memoria RAM pentru un task care are contextul pe 16 biți?
4. Se da un sistem cu trei task-uri. Fiecare task are alocat cate un context de dimensiune 10 octeți. Să se definească structura specifică task-ului și să se construiască administratorul de task-uri ținând cont de priorități. Organizarea task-urilor este următoarea:

Task	P	T	C
1	1	10	2
2	2	10	3
3	3	10	4

